EPFC

PRO
Programmation orientée objet

Leçon 3

Réintroduction à IntelliJ

Introduction aux tests unitaires

Comparaison d'objets

EPFC

Réintroduction à  JetBrains Intellij IDEA

JetBrains IntelliJ IDEA

EPFC

IntelliJ IDEA est un environnement de développement intégré (IDE) conçu pour aider les développeurs à écrire des programmes.

On y trouve donc certaines fonctionnalités pratiques :

- Autocomplétions,
- Débogage,
- Documentation intégrée,
- Refactorisation,
- Support au travail collaboratif et gestion de version (*git* voir TGPR),
- …

Outils de refactorisation

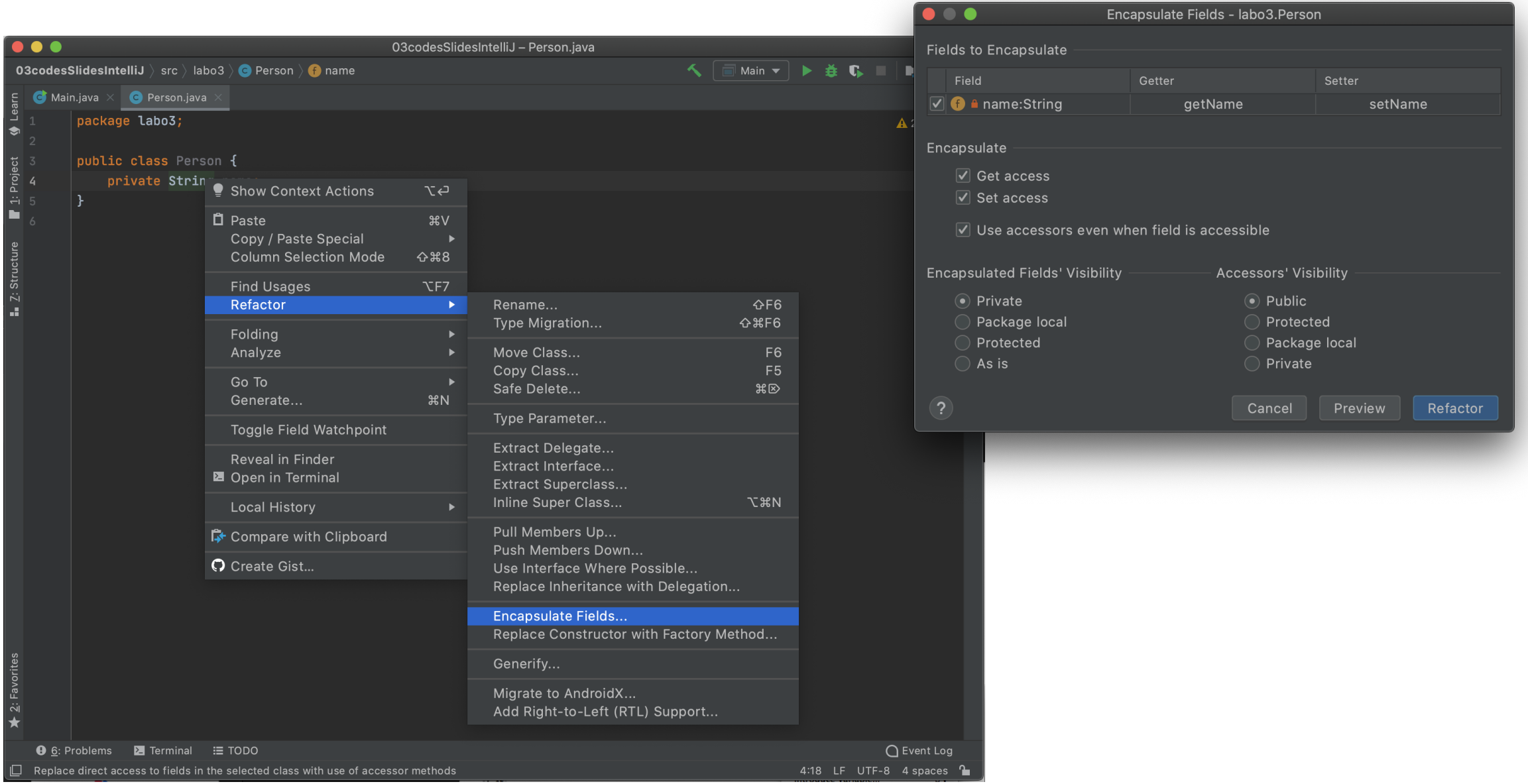
EPFC

La **refactorisation** (refactoring, réusinage) de code consiste à retravailler le code source sans ajouter de fonctionnalité mais en améliorant sa lisibilité.

IntelliJ offre des fonctions qui aident à cela :

- Générer des accesseurs
- Extraire du code dans une méthode
- Renommer un membre
- …

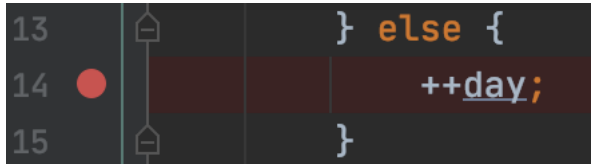
Générer des accesseurs



Utiliser le débogueur

Un **point d'arrêt** est un point placé à un endroit spécifique d'un programme qui permet d'indiquer à un débogueur de suspendre l'exécution lorsqu'il atteindra cet endroit.

Pour fixer un point d'arrêt avec IntelliJ, il suffit de cliquer à droite du numéro de ligne voulu. Un disque rouge apparaît.



Ensuite, il faut lancer le projet en **mode debug**  et l'exécution sera suspendue lors de l'arrivée à la ligne marquée par un point d'arrêt.

Lors de cette suspension, il est possible, entre autre, de consulter graphiquement l'état des variables.

Quelques raccourcis clavier

IntelliJ permet d'accélérer l'écriture de portions de code courantes :

Exemples :

`ALT + ENTER` sur un mot montre des actions contextuelles courantes (importer, faire un test unitaire…)

Taper « `sout` » puis TAB écrit `System.out.println();`

Taper « `psvm` » puis TAB crée une méthode `main();`

…

Documentation :

https://www.jetbrains.com/help/idea/mastering-keyboard-shortcuts.html

EPFC

Introduction aux tests unitaires

Tester le fonctionnement d'une unité

Test unitaires

EPFC

Tests manuels :

Jusqu'à présent, après avoir écrit une méthode, vous écrivez probablement quelques appels dans la méthode main pour tester votre nouvelle méthode.

Mais vous détruisez ce code ensuite…

Tests unitaires :

Un test unitaire est un procédé permettant de s'assurer du fonctionnement correct d'une partie déterminée d'un programme.

Cela se matérialise par de petites méthodes de tests utilisant un *framework* comme par exemple JUnit.

But

Formaliser et automatiser les tests

Avantages

Les tests peuvent être exécutés à chaque modification

Cela permet de vérifier que l'on ne « casse » rien

Framework JUnit

JUnit est un framework de tests unitaires pour Java.

Dans ce cours, nous utiliserons JUnit 4 car les versions suivantes nécessitent des notions que nous verrons plus tard.

```java
public class DateTest {

    //ce test "passe" si tous les assertEquals sont vérifiés
    @Test
    public void testSimpleIncrement() {
        Date d = new Date(1, 1, 2024);
        d.increment();
        assertEquals(2, d.getDay());
        assertEquals(1, d.getMonth());
        assertEquals(2024, d.getYear());
    }

    //ce test "passe" si une RuntimeException est déclenchée
    @Test(expected = RuntimeException.class)
    public void testConstructorInvalidMonth() {
        new Date(1, -1, 2024);
    }
}
```

Une bonne pratique de programmation (Test-Driven Development) consiste à écrire des tests avant d'implémenter de nouvelles fonctionnalités ou lors de la découverte d'un bug.
Et ensuite, de développer jusqu'à ce que l'ensemble des tests passent.

Remarque
Les tests ne servent qu'à vérifier le comportement attendu des méthodes non privées d'une classe.

EPFC

Comparaison d'objets

Deux objets sont ils identiques ?

Dans quel ordre se trouvent deux objets ?

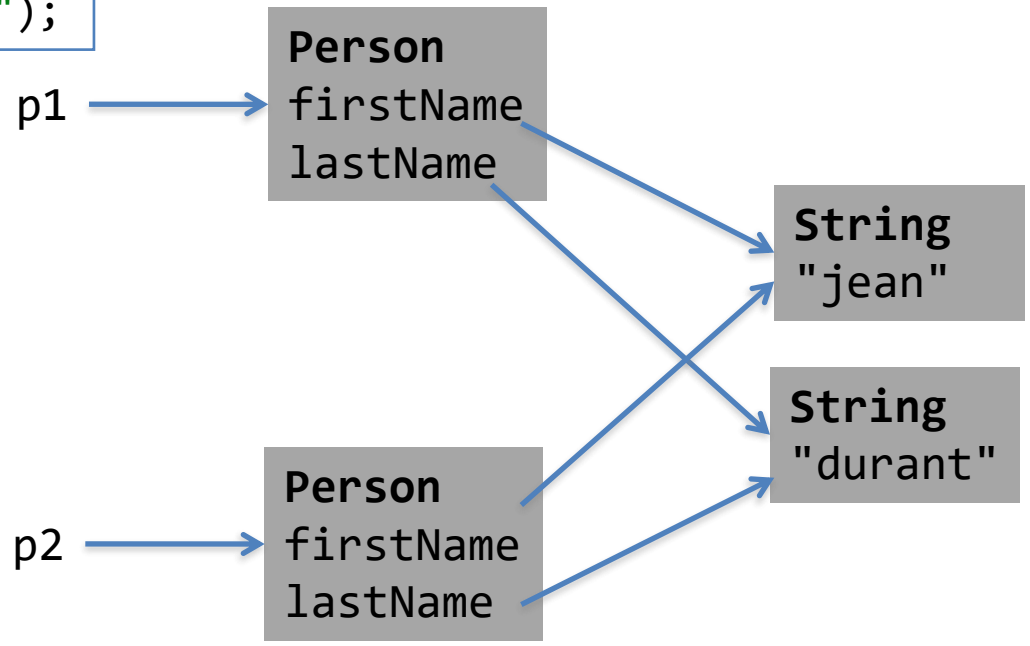
Rappel : les objets sont de type référence

```
Person p1 = new Person();
Person p2 = new Person();

p1.firstName = "jean";
p2.firstName = p1.firstName;

p1.lastName = "durant";
p2.lastName = p1.lastName;

if(p1 == p2)
    System.out.println("Ce message n'est jamais affiché");
```



Mais alors, comment faire pour considérer ces deux objets comme étant identiques ?

La méthode **equals**

`public boolean equals(Object o)` est la méthode Java standard pour déterminer si deux objets sont identiques.

Comme `toString` et d'autres, **`equals`** est définie dans la classe `Object`, c'est à dire qu'elle est « héritée » (voir plus loin) dans toute classe.

Le comportement par défaut de la méthode **`equals`** est de comparer les références (identique à ==).

Si l'on veut un comportement différent, il faut redéfinir cette méthode.

Redéfinir la méthode equals

```
public class Person {
    private String firstName, lastName;

    public boolean equals(Object o) {
        if(o instanceof Person) {
            Person p = (Person)o;
            return (p.firstName == this.firstName &&
                    p.lastName == this.lastName);
        }
        return false;
    }
}
```

Attention, la méthode equals Java reçoit un `Object` (pas une `Person`).

L'opérateur `instanceof` permet de tester le type d'un objet.
Exemple : `if(o instanceof Person)`

`((Person)o)` convertit localement l'objet `o` en `Person`. Cela s'appelle le transtypage ou le *cast*. Soyez prudent, si vous ne faites pas attention, des erreurs peuvent survenir à l'exécution.

Une erreur s'est glissée dans ce transparent. La voyez-vous ?

Comparer deux objets

La méthode **`compareTo`** permet de comparer des objets selon un **ordre** donné.

```
public class Person {
    public int age;

    public int compareTo(Person other) {
        return this.age - other.age;
    }
}
```

`p1.compareTo(p2)` renvoie

`0` si p1 a le même âge que p2

un négatif quelconque si p1 est plus jeune que p2 (« p1 < p2 »)

un positif quelconque si p1 est plus âgé que p2 (« p1 > p2 »)

```
if(p1.compareTo(p2) > 0) { // """p1 > p2""", p1 après p2
    System.out.println(p1 + " plus agé que " + p2);
}
```